



Lecture 05

Describing Web Resources

RDF- RESOURCE DESCRIPTION FRAMEWORK

1. Introduction

The World Wide Web (WWW) – an information system enables content sharing, information exchange and communication.

A webpage is written in HTML to communicate about the structure of information for human consumption.

Semantic Web requires more functionalities: A data model that

- can be used by multiple applications, not just describing documents for people
- is domain independent so that people of different backgrounds can use it

Two XML Problems

- How do you encode the piece of knowledge
“The book FOST is published by CRC Press”
- `<book>`
 `<title>FOST</title>`
 `<publisher>CRC Press</publisher>`
 `</book>`
- `<publisher>`
 `<name>CRC Press</name>`
 `<book><title>FOST</title><book>`
 `</publisher>`
- etc.

Two XML Problems

- Merging trees is rather cumbersome and the result isn't always clear.
 - `<publisher>`
 `<name>CRC Press</name>`
 `<book><title>FOST</title><book>`
 `</publisher>`
 - `<book>`
 `<title>Semantic Web</title>`
 `<publisher>Springer</publisher>`
 `</book>`

Resource Description Framework (RDF)

Originally for describing metadata for web pages, but has grown beyond that.

Structured information.

Universal, machine-readable data exchange format.

Is designed to be read and understood by computers.

Resource Description Framework (RDF)

Domain independent data model.

Built by *statements* – each statement is a *triple*.

Fundamental concepts – resources, properties, statements, and graphs.

RDF: Examples of Use

- Describe properties for shopping items, such as price and availability.
- Describe time schedules for web events.
- Describe information about web pages (content, author, created and modified date).
- Describe content and rating for web pictures.
- Describe content for search engines.
- Describe electronic libraries.

RDF and The Semantic Web: Semantic Web Vision

Web	RDF
Web information has exact meaning.	RDF uses URIs to identify resources.
Web information can be understood and processed by computers.	RDF describes resources with properties and property values.
Computers can integrate information from the web.	

RDF Concept: Resources

A resource = an object/a “thing” to talk about.

Examples: authors, apartments, movie, fruit, etc.

Every resource has a URI (Uniform Resource Identifier). An example of URI is a URL (Uniform Resource Locator/web address).

A URL specifies the location of a resource on the internet, while a URI can be used to identify any type of resource, not just those on the internet. URI is usually used in XML, tag library files, and other files, such as JSTL and XSTL.

RDF Concept: Properties

Describe relations between other resources.

E.g., written by, father of.

Like all resources, properties are also identified by URIs.

RDF Concept: Statement

A triple - consists of a resource, a property and a value.

The value can either be a resource or a literal.

Literal – atomic value (e.g., number, string, date).

RDF Concept: Statement (Example)

“TRX is located in Kuala Lumpur”.

Each of the three parts of the statement can be identified by a URI.

<<http://www.dbkl.gov.my/ontology/commercials.ttl#TRX>>

<<http://dbpedia.org/ontology/location>>

<<http://dbpedia.org/resource/KualaLumpur>>.

What is Resource Description Framework (RDF) in AI?

- A directed graph composed of triple statements.
- A semantic net in the AI community.
- An RDF graph:
 - 1) a node for the subject,
 - 2) an arc that goes from a subject to an object for the predicate, and
 - 3) a node for the object.

RDF Concept: Graph

Graph as list of (node-edge-node) triple/statement.

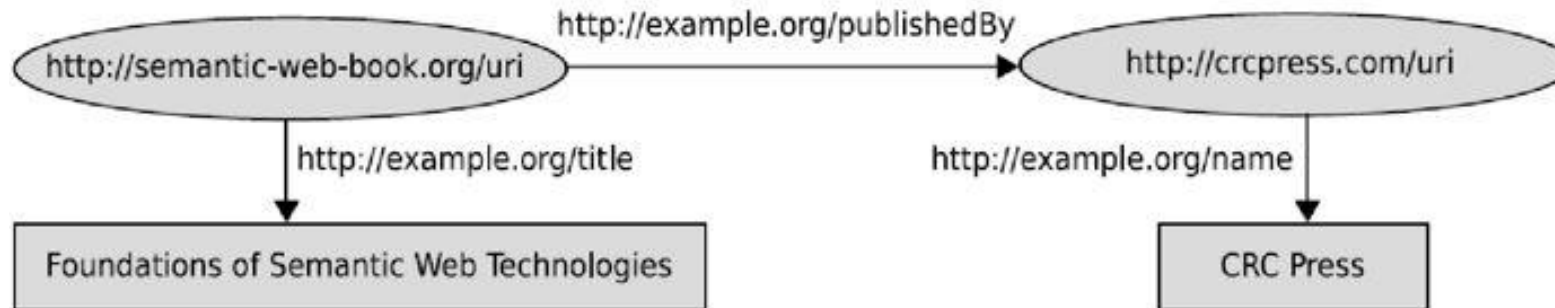


(borrowed from linguistics)

- In the subject : URIs and empty nodes
- In the predicate: URIs (usually called *properties*)
- In the object: URIs and empty nodes and literals

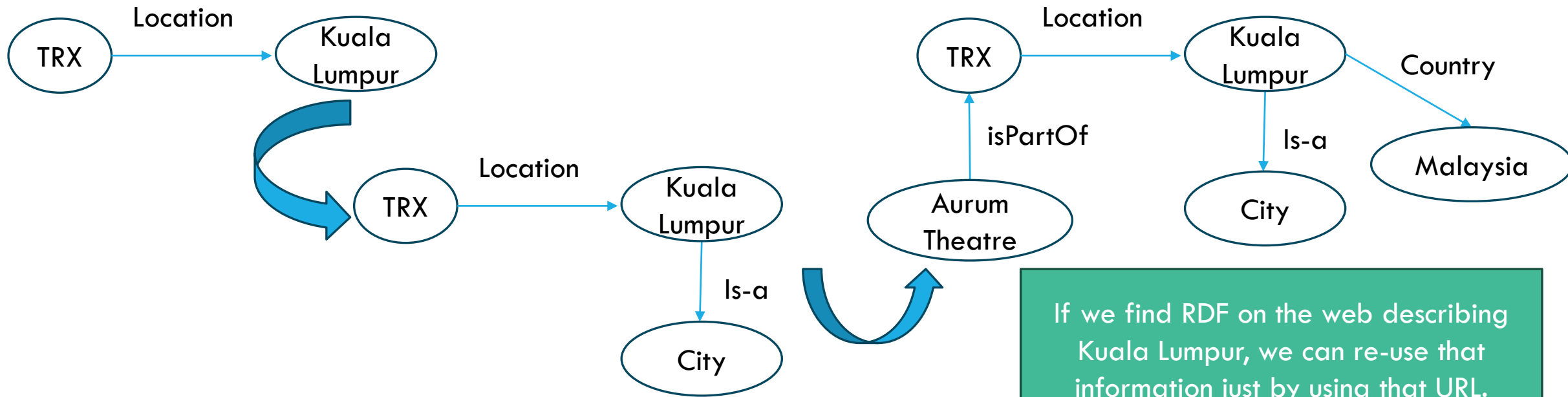
Literal

- for representing data values
- encoded as strings
- interpreted by means of datatypes
- literals without datatype are treated the same as strings



RDF Concept: Graph

Graphs are expandable.



If we find RDF on the web describing Kuala Lumpur, we can re-use that information just by using that URL.
- Linked Data principles

RDF Concept: Graph

Graphs are expandable.

Created in a distributed fashion by multiple different participants just by using the same URLs → *Web of Data* allows re-use of knowledge.

Linked Data Principles

A set of best practices for publishing structured data on the web.

1. Use URLs as names for things.
2. Use HTTP URLs so that people can look up those names.
3. When someone looks up a URI, provide useful information.
4. Include links to other URIs so that they can discover more things.

Dealing with Richer Predicates

A triple (x, P, y) = a logical formula $P(x, y)$

where the binary predicate (property) P relates the object x to the object y .

Question: if a predicate has more than two arguments, e.g., $P(X, Y, Z)$?

X is the broker in a home sale between seller Y and purchaser Z .

Dealing with Richer Predicates (*n*-ary relationships)

X is the broker in a home sale between seller Y and purchaser Z.

broker(X,Y,Z)?

Use a number of binary predicates.

Introduce an auxiliary resource *home-sale* and binary predicates *broker*, *seller* and *purchaser*.

broker(home-sale, X)

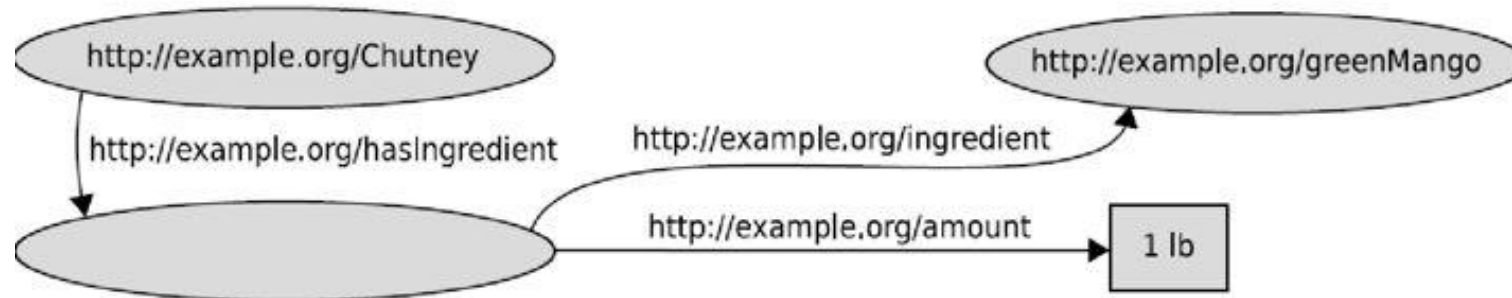
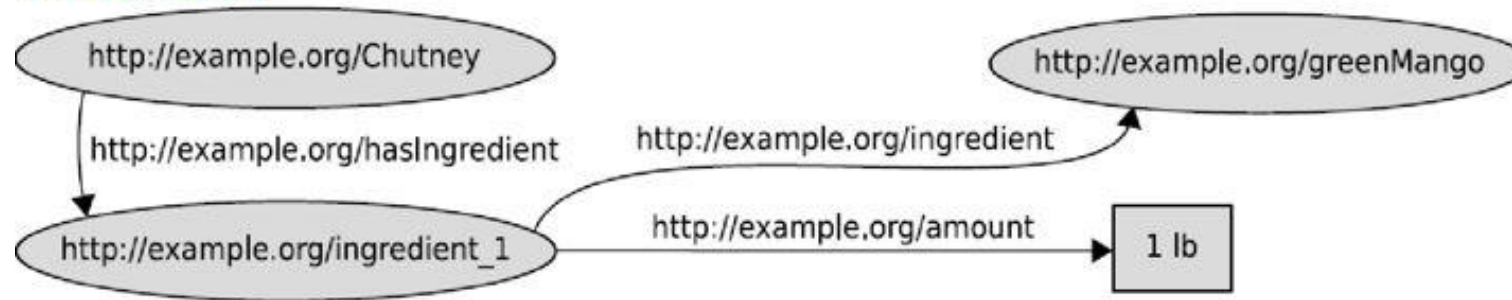
seller(home-sale, Y)

purchaser(home-sale, Z)

The idea of using auxiliary resource
is called as reification.

Empty Nodes (Blank Nodes)

Also do not refer to resources that must be described explicitly/
have no URI!



Empty Node Syntax

```
<rdf:Description rdf:about="http://example.org/Chutney">
  <ex:hasIngredient rdf:nodeID="id1" />
</rdf:Description>
<rdf:Description rdf:nodeID="id1">
  <ex:ingredient rdf:resource="http://example.org/greenMango" />
  <ex:amount>1lb</ex:amount>
</rdf:Description>
```



Blank node

shortcut:

```
<rdf:Description rdf:about="http://example.org/Chutney">
  <ex:hasIngredient rdf:parseType="Resource">
    <ex:ingredient rdf:resource="http://example.org/greenMango" />
    <ex:amount>1lb</ex:amount>
  </ex:hasIngredient>
</rdf:Description>
```

Empty Node Syntax

```
<rdf:Description rdf:about="http://example.org/Chutney">
  <ex:hasIngredient rdf:nodeID="id1" />
</rdf:Description>
<rdf:Description rdf:nodeID="id1">
  <ex:ingredient rdf:resource="http://example.org/greenMango" />
  <ex:amount>1lb</ex:amount>
</rdf:Description>
```

Turtle:

```
@prefix ex: <http://example.org/> .
ex:Chutney    ex:hasIngredient    _:id1 .
_:id1         ex:ingredient    ex:greenMango; ex:amount    "1lb" .
```

Empty Node Syntax

```
@prefix ex: <http://example.org/> .  
ex:Chutney      ex:hasIngredient _:id1 .  
_:id1           ex:ingredient  ex:greenMango;    ex:amount  "1lb" .
```

shortcut:

```
@prefix ex: <http://example.org/> .  
ex:Chutney      ex:hasIngredient  
                [ ex:ingredient  ex:greenMango;    ex:amount  "1lb" ] .
```


2. RDF Syntaxes

Standard machine interpretable syntaxes:

- Terse RDF Triple Language (Turtle)
- RDF/XML
- RDFa

Terse RDF Triple Language (Turtle)

Text-based syntax. File “.ttl”

Follow triple – the subject, property, and object appear in order, ends with a full-stop.

Example:

```
<http://www.dbkl.gov.my/ontology/commercials.ttl#TRX>
```

```
<http://dbpedia.org/ontology/location>
```

```
<http://dbpedia.org/resource/KualaLumpur>.
```

Turtle: Literals

Values are enclosed in quotes.

Data Type	Example
string	"CRC Press"
integer	"2"^^<http://www.w3.org/2001/XMLSchema#integer>
decimal	"2.13"^^<http://www.w3.org/2001/XMLSchema#decimal>
date	"2024-04-23"^^<http://www.w3.org/2001/XMLSchema#date>
time	"16:00:00"^^<http://www.w3.org/2001/XMLSchema#time>
date with a time	"2024-04-23T16:00:00"^^<http://www.w3.org/2001/XMLSchema#dateTime>

Turtle: Literals (Example)

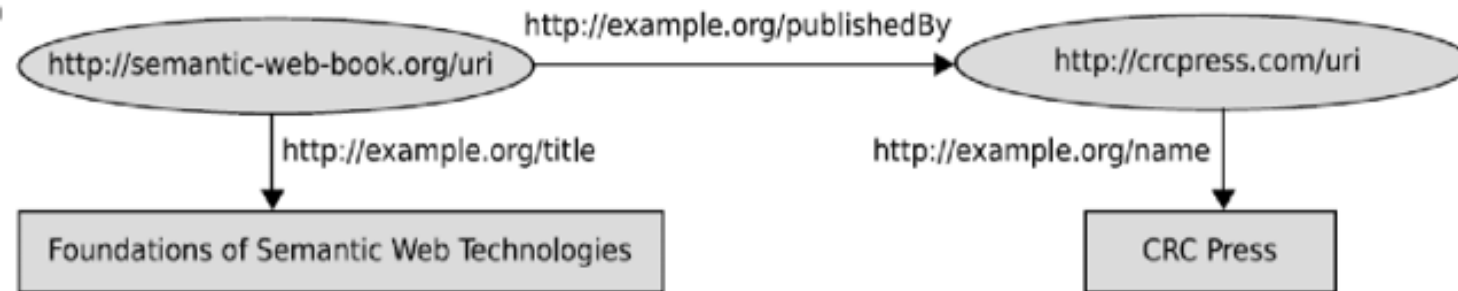
TRX has 106 floors.

`<http://www.dbkl.gov.my/ontology/commercials.ttl#TRX>`

`<http://www.dbkl.gov.my/ontology/commercials.ttl#hasNumberOfFloors>`

`"106"^^<http://www.w3.org/2001/XMLSchema#integer>.`

Turtle: Literals (Example)



```
<http://semantic-web-book.org/uri>
  <http://example.org/publishedBy>  <http://crcpress.com/uri> .
<http://semantic-web-book.org/uri>
  <http://example.org/title>
    "Foundations of Semantic Web Technologies" .
<http://crcpress.com/uri>
  <http://example.org/name>      "CRC Press" .
```

Turtle: Abbreviations

URIs are abbreviated using prefixes of the form “prefix:” without enclosed in angular brackets.

prefix:name is also known as *qualified names* (Qnames).

Benefit: easy to read for human readers.

Turtle: Abbreviations

A URL could be abbreviated and termed with a namespace.

The `@prefix` to define short stand-ins for the namespace. E.g.,

`@prefix building: <http://www.dbkl.gov.my/ontology/commercials.ttl#>.`

`@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.`

`building:TRX    building:hasNumberOfFloors    "106"^^<xsd:integer>.`

Turtle: Abbreviations (Example)

@prefix building: <http://www.dbkl.gov.my/ontology/commercials.ttl#>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

building:TRX building:hasNumberOfFloors "106"^^<xsd:integer>.

Turtle: Abbreviations (Example)

Let say another statement (triple) is added.

“TRX is located in Kuala Lumpur”

@prefix building: <http://www.dbkl.gov.my/ontology/commercials.ttl#>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

@prefix dbpedia: <http://dbpedia.org/resource/>.

@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.

building:TRX building:hasNumberOfFloors “106”^^<xsd:integer>.

building:TRX dbpedia-owl:location dbpedia:KualaLumpur.

Turtle: Abbreviations (Example)

Let say another statement (triple) is added.

“TRX is located in Kuala Lumpur”

@prefix building: <http://www.dbkl.gov.my/ontology/commercials.ttl#>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

@prefix dbpedia: <http://dbpedia.org/resource/>.

@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.

building:TRX building:hasNumberOfFloors “106”^^<xsd:integer>;

dbpedia-owl:location dbpedia:KualaLumpur.

Turtle: Abbreviations (Example)

```
@prefix book: <http://semantic-web-book.org/> .
@prefix ex: <http://example.org/> .
@prefix crc: <http://crcpress.com/> .

book:uri    ex:publishedBy    crc:uri .
book:uri    ex:title          "Foundations of Semantic Web Technologies" .
crc:uri     ex:name           "CRC Press" .
```

- grouping of triples with the same subject
- grouping of triples with same subject and predicate

```
@prefix book: <http://semantic-web-book.org/> .
@prefix ex: <http://example.org/> .
@prefix crc: <http://crcpress.com/> .

book:uri    ex:publishedBy    crc:uri ;
            ex:title          "Foundations of Semantic Web Technologies" .
crc:uri     ex:name           "CRC Press"
```

Turtle: Abbreviations (Example)

Simple triples

```
<http://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Spiderman" .  
<http://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Человек-паук"@ru .
```

Equivalent to:

Type a comma at the end of a statement if both a subject and predicate are used repeatedly.

```
<http://example.org/#spiderman> <http://xmlns.com/foaf/0.1/name> "Spiderman", "Человек-паук"@ru .
```

Turtle: Abbreviations (Example)

Can also abbreviate common data types. E.g., numbers can be written without quotes.

@prefix building: <http://www.dbkl.gov.my/ontology/commercials.ttl#>.

@prefix xsd: <http://www.w3.org/2001/XMLSchema#>.

@prefix dbpedia: <http://dbpedia.org/resource/>.

@prefix dbpedia-owl: <http://dbpedia.org/ontology/>.

building:TRX building:hasNumberOfFloors 106;

dbpedia-owl:location dbpedia:KualaLumpur.

RDF/XML

Another syntax for RDF - An encoding of RDF in the XML language.

Can use together with existing XML processing tools.

The main elements of RDF are the root element, `<RDF>`, and the `<Description>` element, which identifies a resource.

RDF/XML

Subject:

Denoted by the `rdf:about` within an `rdf:Description` element (enclosed in brackets)

Predicate and object:

Related to the subject that is enclosed in the `rdf:Description` element

RDF/XML (Example)

```
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:feature="https://www.linkeddatatools.com/clothing-features#">

  <rdf:Description rdf:about="https://www.linkeddatatools.com/clothes#t-shirt">

    <feature:size>12</feature:size>

  </rdf:Description>

</rdf:RDF>
```

xmlns:namespace (Namespace is used through the XML namespace construct.

RDF/XML (Example)

1 predicate

```
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:feature="https://www.linkeddatatools.com/clothing-features#">

  <rdf:Description rdf:about="https://www.linkeddatatools.com/clothes#t-shirt">

    <feature:size>12</feature:size>

  </rdf:Description>

</rdf:RDF>
```

RDF/XML (Another Example)

Add one more predicate: the color of the T-shirt.

```
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:feature="https://www.linkeddatatools.com/clothing-features#">

  <rdf:Description rdf:about="https://www.linkeddatatools.com/clothes#t-shirt">

    <feature:size>12</feature:size>
    <feature:color rdf:resource="https://www.linkeddatatools.com/colors#white"/>

  </rdf:Description>

</rdf:RDF>
```

The <rdf:RDF> Element

It is the root element of an RDF document.

Defines the XML document as an RDF document.

Contains a reference to the RDF namespace.

```
<?xml version="1.0"?>
```

```
<rdf:RDF
```

```
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
```

```
    ...Description goes here...
```

```
</rdf:RDF>
```

The **xmlns:rdf** namespace, specifies that elements with the **rdf prefix** are from the namespace `"http://www.w3.org/1999/02/22-rdf-syntax-ns#"`

The <rdf:Description> Element

Contains the description
of the resource identified
by
the **rdf:about** attribute.

Identifies a resource with
the about attribute.

Contains elements that
describe the resource

```
<?xml version="1.0"?>
```

```
<rdf:RDF
```

```
xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
xmlns:cd="http://www.recshop.fake/cd#">
```

```
<rdf:Description
```

```
rdf:about="http://www.recshop.fake/cd/Empire_Burlesque">
```

```
<cd:artist>Bob Dylan</cd:artist>
```

```
<cd:country>USA</cd:country>
```

```
<cd:company>Columbia</cd:company>
```

```
<cd:price>10.90</cd:price>
```

```
<cd:year>1985</cd:year>
```

```
</rdf:Description>
```

```
</rdf:RDF>
```

<cd:artist>, **<cd:country>**,
<cd:company>, etc. are properties of the
resource

Properties as Attributes

The property elements can be defined as attributes (instead of elements).

```
<?xml version="1.0"?>

<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:cd="http://www.recshop.fake/cd#">

  <rdf:Description
    rdf:about="http://www.recshop.fake/cd/Empire Burlesque">
    <cd:artist>Bob Dylan</cd:artist>
    <cd:country>USA</cd:country>
    <cd:company>Columbia</cd:company>
    <cd:price>10.90</cd:price>
    <cd:year>1985</cd:year>
  </rdf:Description>

</rdf:RDF>
```

Properties as Resources

The property elements can also be defined as resources.

```
<?xml version="1.0"?>

<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:cd="http://www.recshop.fake/cd#">

  <rdf:Description
    rdf:about="http://www.recshop.fake/cd/Empire_Burlesque">
    <cd:artist rdf:resource="http://www.recshop.fake/cd/dylan" />
    ...
    ...
  </rdf:Description>

</rdf:RDF>
```

The property artist does not have a value, but a reference to a resource containing information about the artist.

RDFa

(Resource Description Framework in Attributes)

To describe/mark up the content of HTML webpages.

RDFa embeds RDF within the attributes of HTML tags.

Basic RDFa syntax schema:

The vocab attribute defines the vocabulary that forms the basis of tagging elements with RDFa

With the typeof attribute, elements are assigned certain themes (i.e. 'types') according to the selected vocabulary.

div tag creates a block-level container that groups elements

span tag creates a inline container within a block-level container

```
<div vocab="http://Schema.org/" typeof="Schema">  
  <span property="Characteristic">text element</span>  
</div>
```


RDFa (Example)

HTML code of a mailing address

```
<p>  
  Google Inc.<br>  
  P.O. Box 1234<br>  
  Mountain View, CA<br>  
  94043<br>  
  United States<br>  
</p>
```

RDFa markup of a mailing address

```
<p vocab="http://Schema.org/" typeof="PostalAddress"><br>  
  <span property="name">Google Inc.</span><br>  
  P.O. Box <span property="postOfficeBoxNumber">1234</span><br>  
  <span property="addressLocality">Mountain View</span>,<br>  
  <span property="addressRegion">CA</span><br>  
  <span property="postalCode">94043</span><br>  
  <span property="addressCountry">United States</span><br>  
</p>
```



```
<p vocab="http://Schema.org/" typeof="PostalAddress"><br>
```

In line 01, the HTML tag, <p>, works as a basis for the RDFa attributes ‘vocab’ and ‘typeof’.

Programs that read code tagged in this way are able to recognize two **distinct bits of information**: that all the entities contained within the <p> tag are in accordance with Schema.org’s vocabulary *and* that they have also been assigned to the type, ‘PostalAddress’.

As per Schema.org, each type can be assigned specific properties. With regard to the given example, the RDFa attribute ‘property’ is implemented in order to make the address details **machine readable**.

The address details ‘name’, ‘postOfficeBoxNumber’, ‘addressLocality’, ‘addressRegion’, ‘postalCode’, and ‘addressCountry’ with their corresponding values are tagged as properties of the type, ‘PostalAddress’.

RDFa markup of a mailing address

```
<p vocab="http://Schema.org/" typeof="PostalAddress"><br>  
  <span property="name">Google Inc.</span><br>  
  P.O. Box <span property="postOfficeBoxNumber">1234</span><br>  
  <span property="addressLocality">Mountain View</span>,<br>  
  <span property="addressRegion">CA</span><br>  
  <span property="postalCode">94043</span><br>  
  <span property="addressCountry">United States</span><br>  
</p>
```